Developing Concurrent & Asynchronous Applications

Ensuring High Performance and Responsive Applications

MULTITHREADING

1. Multithreading in Python

- Spawns real, operating-system-level threads of execution inside Python, each with its own call stack but sharing the same process memory.
- The OS scheduler manages context-switching between active threads automatically, with no manual intervention required from the programmer.
- The Global Interpreter Lock (GIL) allows only one thread to execute Python bytecode at a time, limiting CPU-bound work to a single core even with multiple threads running.
- Threads are highly effective for offloading blocking I/O work, such as network requests or disk file operations, since the GIL is released while a thread waits.
- Standard threads carry significant memory overhead compared to lightweight coroutines, and must be explicitly joined so the main program waits for them to finish.

```python
import threading
import time

def worker():
    print("Thread started")
    time.sleep(0.5)
    print("Thread done")

def main():
    # Create thread object
    t = threading.Thread(target=worker)

    # Start the thread execution
    t.start()

    # Block main thread until t completes
    t.join()
    print("Main finished.")

if __name__ == "__main__":
    main()
```

NOTE

The target function runs in an independent OS thread. Calling start() schedules it, and join() blocks the main thread until it completes, ensuring orderly workflow cleanup.

MULTITHREADING

The Main Thread vs. The Target Thread

The Main Thread

- Starts automatically when you run a Python script.
- Executes your primary code sequentially from top to bottom.
- Spawns and manages secondary threads.
- Handles final program shutdown and cleanup.

The Target Thread

- Created manually using threading.Thread(target=my_function).
- Runs independently alongside the main thread.
- Executes only the specific code inside its assigned “target” function.
- Destroys itself automatically once that function finishes executing.

NOTE

Every Python program has exactly one main thread, created automatically at startup. Target threads are created explicitly by the developer and exist only for the lifetime of the function they were assigned to run.

2. Race Conditions and Thread Safety

- Multiple threads share access to the exact same process memory space, so a variable modified by one thread is immediately visible, and mutable, by every other thread.
- When threads interleave their reads and writes to shared state without protection, the result is state corruption, known as a race condition.
- Simple-looking operations such as counter += 1 are not atomic under the hood; they compile into separate read, modify, and write steps that another thread can interrupt.
- Race conditions are highly non-deterministic and notoriously difficult to debug, since the same code can run correctly hundreds of times before failing unpredictably.
- Any critical section that reads and writes shared state must be protected using Locks or other synchronization primitives to guarantee correctness.

```python
import threading
import time

counter = 0

def increment():
    global counter
    for _ in range(100000):
        temp = counter  # Read
        time.sleep(0)   # Force a context switch here
        counter = temp + 1  # Write

def main():
    threads = [
        threading.Thread(target=increment)
        for _ in range(2)
    ]
    for t in threads: t.start()
    for t in threads: t.join()

    # Expected: 200000
    # Actual: reliably less
    print(f"Counter: {counter}")

if __name__ == "__main__":
    main()
```

NOTE

Two concurrent threads modify an unprotected counter. Splitting the read and write with a forced context switch (time.sleep(0)) guarantees the threads interleave, so the lost updates are reproducible every run rather than left to chance timing.

Synchronizing Threads with a Lock

- threading.Lock() creates a mutual-exclusion lock: only one thread may hold it at a time, and every other thread attempting to acquire it must wait.
- Wrapping the critical section in with lock: acquires the lock on entry and guarantees it is released on exit, even if an exception is raised inside the block.
- The read, the forced context switch, and the write now all happen while the lock is held, so the second thread cannot interleave partway through.
- A thread that calls lock.acquire() while the lock is held simply blocks, resuming only once the holding thread calls lock.release() (handled automatically by with).
- The same interleaving-inducing time.sleep(0) from the previous slide is still present here, but it no longer causes lost updates, since the lock removes the possibility of overlap.

```python
import threading
import time

counter = 0
lock = threading.Lock()

def increment():
    global counter
    for _ in range(100000):
        with lock:
            temp = counter      # Read
            time.sleep(0)       # Still forces a switch...
            counter = temp + 1  # ...but the lock blocks the
            # other thread from entering until release

def main():
    threads = [
        threading.Thread(target=increment)
        for _ in range(2)
    ]
    for t in threads: t.start()
    for t in threads: t.join()

    # Expected: 200000
    # Actual: always 200000
    print(f"Counter: {counter}")

if __name__ == "__main__":
    main()
```

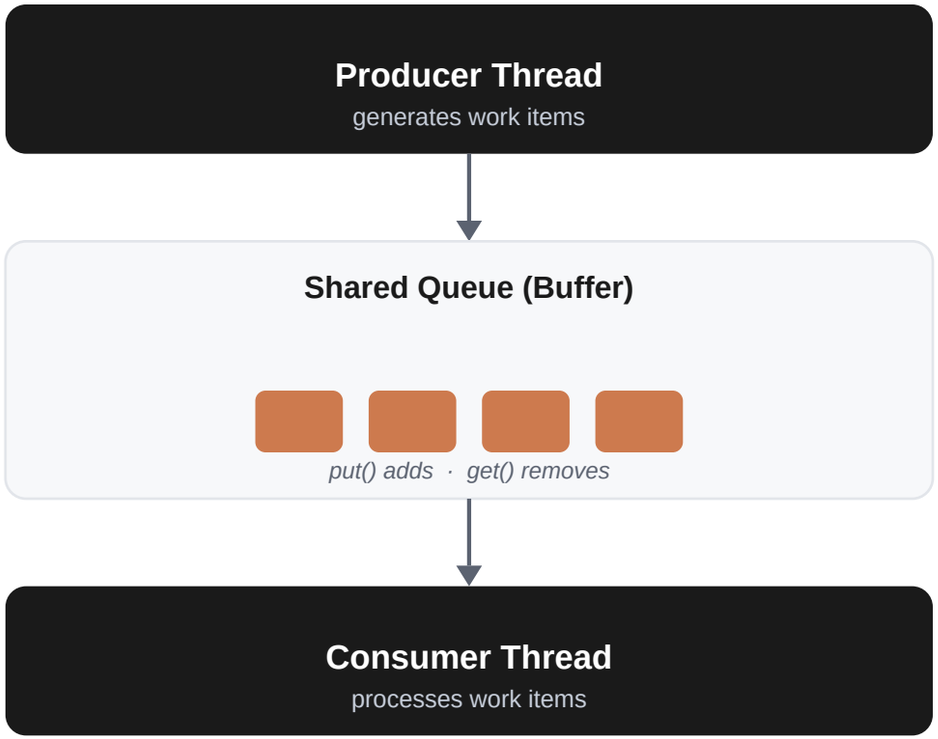
NOTE

The lock turns the read-modify-write sequence into a single atomic step from the other thread's point of view. Even with the deliberate context switch still in the code, the count comes out correct on every run.

MULTITHREADING

What Is the Producer-Consumer Pattern?

- A classic way to structure concurrent work: one or more producer threads generate items, while one or more consumer threads process them.
- Producers and consumers run at their own pace. A shared buffer sits between them so a fast producer doesn’t have to wait on a slow consumer, and vice versa.
- Without protection, a producer writing to that buffer at the same instant a consumer reads from it is exactly the unsynchronized shared-state access that causes the race conditions seen earlier.
- A thread-safe queue built specifically for this job already contains its own internal Lock, so producers and consumers never have to manage synchronization by hand.
- This shape shows up everywhere in real software: request handling, logging pipelines, background job queues, and data-processing pipelines all follow the same pattern.



NOTE

The queue on the next slide is doing the same job as the Lock from the previous slide, just packaged specifically for safely handing items from one thread to another.

3. Producer-Consumer with Threads

- Bounded buffers coordinate speed differences between producers and consumers, preventing a fast producer from overwhelming a slow consumer.
- queue.Queue implements its own internal locks, making put() and get() safe to call concurrently from multiple threads without any extra synchronization code.
- Thread execution suspends automatically when the queue is completely full (on put) or completely empty (on get), so no manual polling or busy-waiting is needed.
- Putting a sentinel value such as None onto the queue is a simple, explicit way to signal to a worker thread that no more work is coming.
- Worker threads should check for and break on the sentinel, ensuring the consumer exits cleanly instead of blocking forever waiting for the next item.

```python
import queue, threading, time

q = queue.Queue(maxsize=3)

def producer():
    for i in range(5):
        print(f"Producing: {i}")
        q.put(i)  # Blocks if full
        time.sleep(0.1)
    q.put(None)  # Sentinel

def consumer():
    while True:
        item = q.get()  # Blocks if empty
        if item is None: break
        print(f"Consumed: {item}")
        time.sleep(0.2)

def main():
    p = threading.Thread(target=producer)
    c = threading.Thread(target=consumer)
    p.start()
    c.start()
    p.join()
    c.join()

if __name__ == "__main__":
    main()
```

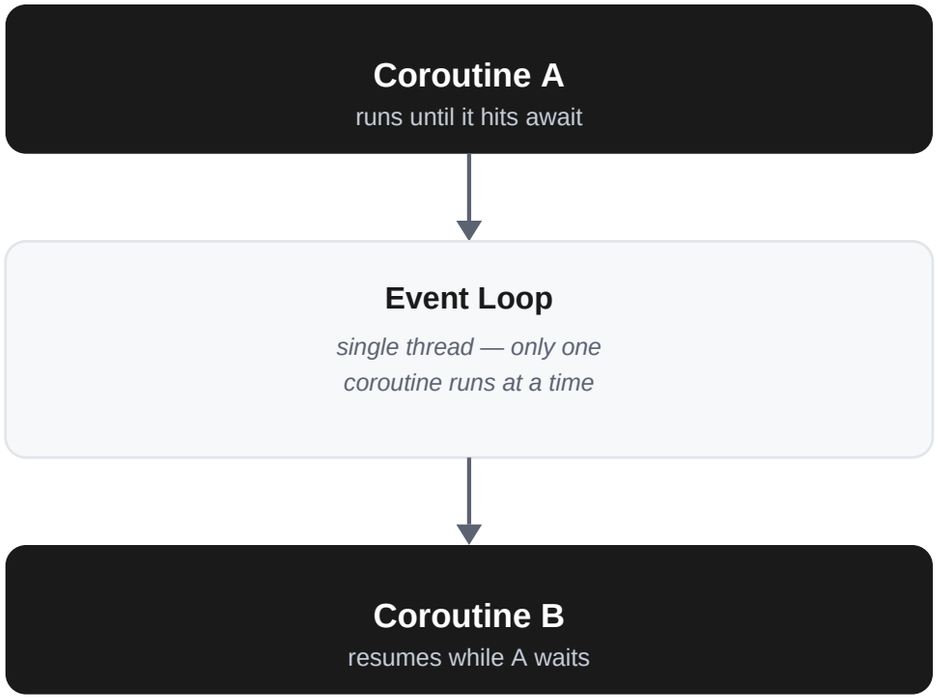
NOTE

Safe thread coordination using a bounded queue. The producer sleeps during writes, and the consumer blocks dynamically on empty buffers until a None sentinel signals clean shutdown.

ASYNCIO BASICS

Key Asyncio Terms vs. Multithreading

- Coroutine: a function defined with async def that can pause partway through and resume later from exactly where it left off, instead of running start-to-finish in one go.
- await: marks the exact point where a coroutine is willing to pause and hand control back, rather than blocking the whole program while it waits.
- Event Loop: the single-threaded scheduler that runs one coroutine at a time, switching to another only when the current one hits an await.
- Multithreading uses multiple OS threads that the operating system can interrupt at almost any point (preemptive); asyncio uses one thread where coroutines only pause exactly where they say await (cooperative).
- Because switching only ever happens at an explicit await, asyncio avoids many of the surprise race conditions seen earlier — though shared state touched across an await can still need protecting, as we will see with asyncio.Lock.



NOTE

Multithreading: many threads, the OS decides when to switch. Asyncio: one thread, the code decides when to switch — only at await.

ASYNCIO BASICS

4. Introduction to asyncio and Coroutines

- Runs a single-threaded, cooperative multitasking engine on a single core, switching between tasks only at points the code explicitly allows.
- Coroutines explicitly yield control back to the event loop using await points, rather than being pre-emptively interrupted the way OS threads are.
- Because coroutines are managed entirely in user space, they eliminate the memory and scheduling overhead that comes with spawning many heavyweight OS threads.
- The event loop coordinates scheduling, resuming a suspended coroutine only once the operation it was waiting on, such as I/O, is actually ready.
- Awaiting a coroutine suspends the calling coroutine at that point, allowing other coroutines to run during the wait instead of blocking the whole program.

```python
import asyncio

async def greet():
    print("Hello...")
    # Yield control to the loop for 0.5s
    await asyncio.sleep(0.5)
    print("...World!")

async def main():
    # Awaiting suspends main until greet is done
    await greet()

if __name__ == "__main__":
    # Handles event loop lifecycle
    asyncio.run(main())
```

NOTE

Executing a basic coroutine using asyncio.run(). Awaiting asyncio.sleep() yields execution control back to the event loop, allowing other concurrent tasks to be multiplexed.

ASYNC CONCURRENCY

5. Concurrency with asyncio.create_task

- create_task() wraps a coroutine in a Task object and immediately schedules it to run on the active event loop, rather than waiting for it to be awaited.
- This enables multiple coroutines to make progress concurrently, even though they are all still executing on a single physical thread.
- Because the task begins running in the background right away, the calling code can continue with other work before the task's result is actually needed.
- A Task behaves like an await-compatible Future: awaiting it later simply retrieves the result once the task has completed.
- Launching tasks upfront and awaiting them afterwards collapses the total wait time to roughly the duration of the single longest task, rather than the sum of all of them.

```python
import asyncio

async def fetch(id_code):
    print(f"Start fetch {id_code}...")
    await asyncio.sleep(0.5)
    print(f"End fetch {id_code}")
    return f"Data {id_code}"

async def main():
    # Schedule concurrently on the active loop
    t1 = asyncio.create_task(fetch("A"))
    t2 = asyncio.create_task(fetch("B"))

    # Await in parallel
    res1 = await t1
    res2 = await t2
    print(f"Results: {res1}, {res2}")

if __name__ == "__main__":
    asyncio.run(main())
```

NOTE

Concurrent task scheduling with create_task. By launching both tasks background-wise first, they run in parallel, reducing execution latency from 1.0 second down to 0.5 seconds.

ASYNC CONCURRENCY

6. Orchestrating with asyncio.gather

- asyncio.gather() takes multiple awaitables and runs them concurrently as a single group, returning once every one of them has completed.
- The list of results is guaranteed to be returned in the same order the awaitables were passed in, regardless of which one actually finishes first.
- This makes gather ideal for managing a dynamic, variable-length list of concurrent operations, such as a batch of network scrapers or API queries.
- Using gather avoids the repetitive boilerplate of manually creating a task for every item and collecting each result individually.

```python
import asyncio

async def process(val):
    await asyncio.sleep(0.1)
    return val * 10

async def main():
    # Run multiple coroutines concurrently
    results = await asyncio.gather(
        process(1),
        process(2),
        process(3)
    )
    # Guaranteed returned in call order
    print(f"Gathered results: {results}")

if __name__ == "__main__":
    asyncio.run(main())
```

NOTE

Concurrently running multiple coroutines using asyncio.gather. The event loop executes them in parallel on a single thread, returning results in their original invocation order.

ASYNC CONCURRENCY

7. Resource Synchronization with Locks

- Even though only one coroutine runs at a time, tasks can still cause race conditions at the exact point an await statement yields control mid-operation.
- asyncio.Lock restricts access to a critical section so that only one coroutine task can hold it, and therefore execute that section, at any given moment.
- Using the async with lock: pattern guarantees the lock is released automatically on exit, even if an exception is raised inside the block.
- Coroutines blocked waiting for the lock suspend dynamically and queue up, resuming in turn once the lock becomes available.
- This prevents interleaving tasks from reading stale or partially updated values from shared memory during an await yield point.

```python
import asyncio

counter = 0
lock = asyncio.Lock()

async def increment():
    global counter
    async with lock:
        val = counter
        # await yields, but lock blocks others
        await asyncio.sleep(0.01)
        counter = val + 1

async def main():
    await asyncio.gather(
        *(increment() for _ in range(5))
    )
    print(f"Counter: {counter}")

if __name__ == "__main__":
    asyncio.run(main())
```

NOTE

Ensuring data safety across await yield points with asyncio.Lock. The lock guarantees that only one task can execute the counter increment critical section at any given moment.

ASYNC CONCURRENCY

8. Asyncio Producer-Consumer Model

- asyncio.Queue manages streaming data pipelines between coroutines, coordinating producers and consumers entirely through async task scheduling.
- Both put() and get() are awaitable, suspending the calling coroutine automatically whenever the buffer is completely full or completely empty.
- Setting a bounded maxsize protects the program from runaway memory growth by applying backpressure when the consumer falls behind the producer.
- task_done() and q.join() work together to let the producer wait until every item it enqueued has actually been processed by a consumer.
- Background consumer tasks that would otherwise run forever are cancelled cleanly with task.cancel() once the queue has been fully drained.

```python
import asyncio

q = asyncio.Queue(maxsize=3)

async def producer():
    for i in range(5):
        await q.put(i)  # Suspends if full
        print(f"Produced: {i}")
        await asyncio.sleep(0.1)

async def consumer():
    while True:
        item = await q.get()  # Suspends if empty
        print(f"Consumed: {item}")
        q.task_done()
        await asyncio.sleep(0.2)

async def main():
    c_task = asyncio.create_task(consumer())
    await producer()
    await q.join()
    c_task.cancel()
    try:
        await c_task
    except asyncio.CancelledError:
        pass
    print("All items processed.")

if __name__ == "__main__":
    asyncio.run(main())
```

NOTE

A non-blocking async pipeline using asyncio.Queue. The producer populates the queue, the consumer processes items in the background, and q.join() coordinates completion.

ASYNC MIXING

9. Integrating Synchronous Blocking Code

- Any synchronous, blocking call executed directly inside a coroutine stalls the single-threaded event loop completely, freezing every other running task.
- asyncio.to_thread() offloads a blocking function call to a separate worker thread, so it can run without stalling the loop.
- The call runs in asyncio's default thread pool executor, keeping the main event loop thread completely free to keep scheduling other coroutines.
- This is essential when integrating legacy synchronous libraries, performing local file-system disk writes, or handling other unavoidably blocking work.
- Wrapping the blocking call this way preserves cooperative concurrency across the rest of the application without needing to rewrite the blocking function itself.

```python
import asyncio
import time

def blocking_write(data):
    # Simulate a blocking synchronous disk write
    print(f"Starting write for {data}...")
    time.sleep(0.5)
    print(f"Finished write for {data}")
    return f"Saved {data}"

async def main():
    # Offload the blocking function safely
    result = await asyncio.to_thread(
        blocking_write, "dataset_v6"
    )
    print(f"Outcome: {result}")

if __name__ == "__main__":
    asyncio.run(main())
```

NOTE

Running synchronous, blocking code safely with asyncio.to_thread. This offloads the slow blocking_write to a separate OS thread, preventing the main event loop from stalling.

ACTIVITIES

8 Activities to consolidate concepts

- Activity 1: Simple Async Greeting — declare and await a basic coroutine that prints a greeting.
- Activity 2: Concurrent Task Scheduling — schedule multiple API request checkers to run in parallel.
- Activity 3: Bulk Gathering of Awaitables — orchestrate a list of scraper coroutines with asyncio.gather.
- Activity 4: Bounded Web Requests with Timeouts — implement asyncio.wait_for to prevent the event loop from hanging.
- Activity 5: State Synchronization with Locks — eliminate race hazards on shared memory writes using asyncio.Lock.
- Activity 6: Throttling Concurrency with Semaphores — restrict parallel API requests down to 2 concurrent slots.
- Activity 7: Offloading Blocking Synchronous Tasks — run heavy CPU-bound factory computations safely in worker threads.
- Activity 8: Producer-Consumer Pipeline — build a complete async ETL pipeline using asyncio.Queue and Semaphores.

```
# Verification Guidelines:
# 1. Run templates from the outbox.
# 2. Complete the # TODO placeholders.
# 3. Test normal completion and error states.
# 4. Verify thread and asyncio behaviors.
# 5. Review official solutions when done.

# Spawning concurrency safely is the core
# goal of high-performance Python architectures!
# Good luck with the activities!
```

NOTE

The 8-step hands-on programming exercises. Completing this progressive lab series systematically builds deep, mechanical understanding of multi-threaded safety and async coordination.